AMD DEVELOPER HACKATHON ACT III · REINVENT COMMERCE

Chirak

Describe a task, give a few examples, get a small fine-tuned model that does that one job on your own server.

Чирак: apprentice. A large model teaches a small one a single craft.

github.com/escapeboy/chirak · MIT

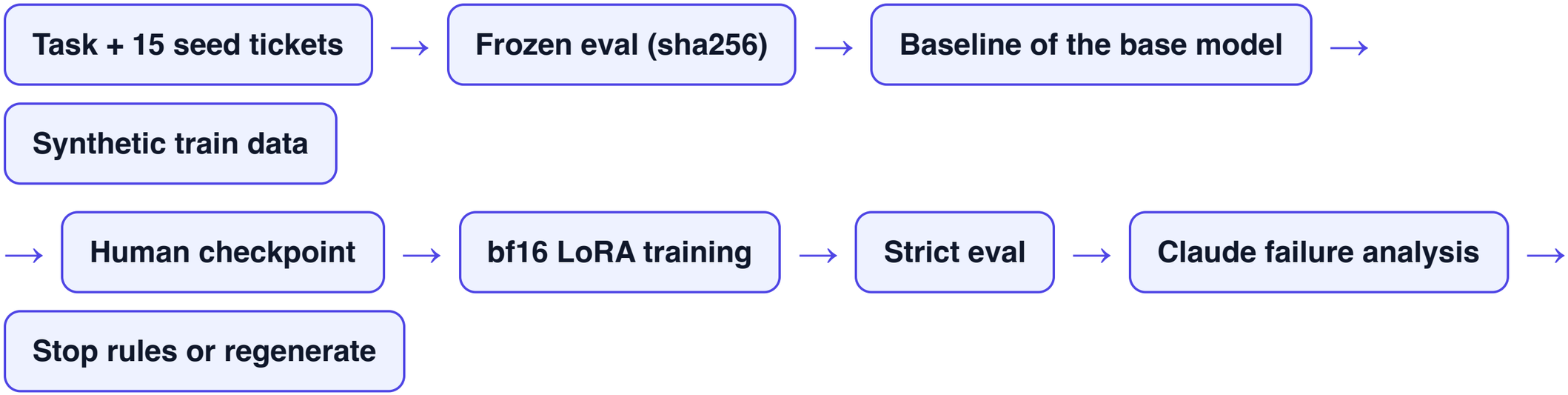
PROBLEM

Shops need a narrow model, not a data science team

- **Narrow, repetitive jobs:** route a support ticket, flag what needs a person, set urgency. A general chatbot API is overkill and costs per call forever.
- **Data stays home:** customer tickets carry names, orders and payment details. In the EU (GDPR) many shops prefer a model on their own server.
- **No ML engineer:** building a fine-tune means data, an honest eval, training and serving. Small merchants cannot staff that.
- **Bulgarian matters:** customers write BG, EN and both in one message.

WHAT IT DOES

One loop, from description to served adapter



The output is a LoRA adapter for Qwen3.5-4B, a public report built from stored results, and a playground that compares the adapter with the base model side by side.

WHY IT IS DIFFERENT

Measured honestly, stopped by rules

Eval first

The eval set is generated and frozen with a sha256 before any training data exists. A reply that is not exactly the JSON object counts as wrong.

No leakage

Train data is written by a different prompt that cannot see eval. Near-copies (bge-m3 cosine ≥ 0.92 or a shared 8-gram) are dropped and counted.

Failure-driven data

Claude reads the confusion matrix and failures and asks for the next slices in strict JSON. Guidance that quotes eval text is discarded.

Deterministic stop

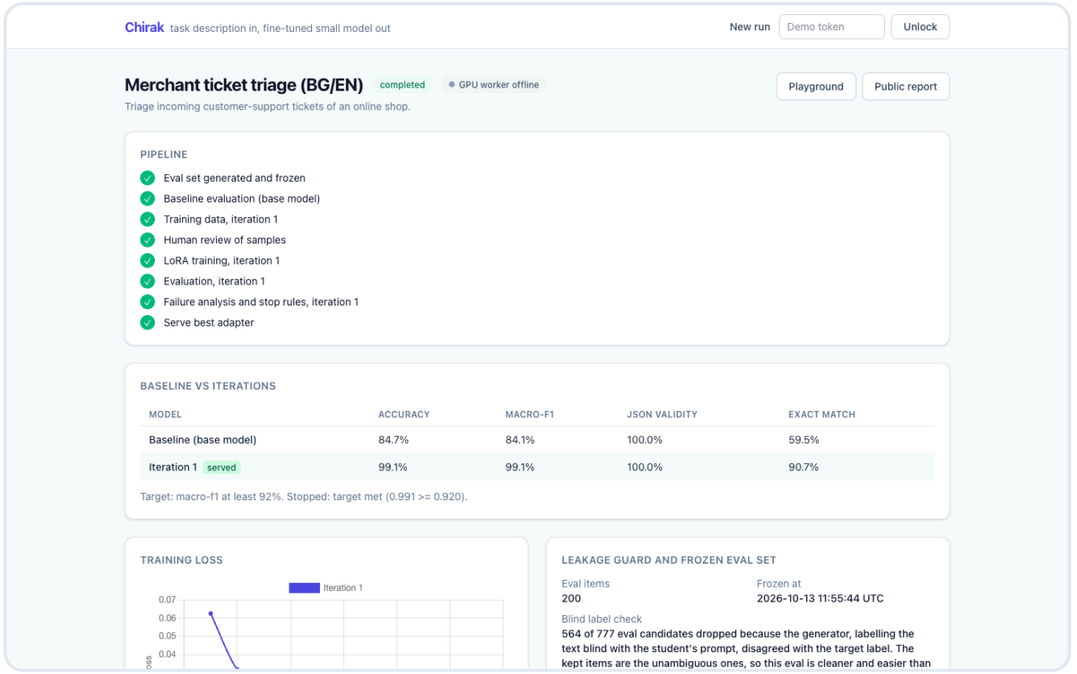
Target met, 3 iterations, under 1 point of gain, or leakage over 30%. In code, not left to the LLM. It also says "not worth fine-tuning".

DEMO RUN · MERCHANT TICKET TRIAGE, BG/EN

Base 0.84 → fine-tuned 0.99 macro-F1

FROZEN EVAL, 215 TICKETS	BASE 4B	+ LORA
macro-F1 (category)	0.84	**0.99**
urgency accuracy	0.87	**0.96**
needs_human accuracy	0.77	**0.95**
all 3 fields right	0.60	**0.91**
valid JSON	100%	100%

Eval and train passed a blind label check by the 27B generator, so this eval holds the clear cases. Unfiltered checks, small samples: 15 seed tickets, all fields 7/15 → 11/15; 5 hold-out tickets, 2/5 → 4/5. Re-scored with `make eval`: adapter 0.9907 in 3/3 runs, base 0.836–0.846.



BUILT ON AMD

Everything runs on one AMD Instinct GPU

STEP	STACK	DEMO RUN ON MI350X
Data generation, Qwen3.8-27B	vLLM 0.31 on ROCm 7.2	1000 train items from 3923 candidates, ~6 min
Leakage guard, bge-m3	PyTorch ROCm	same GPU
LoRA training, Qwen3.5-4B bf16	torch 2.14.1+rocm7.2, peft, trl	1000 examples, 2 epochs, 130 s, 12 GB
Eval and serving	vLLM, runtime LoRA hot-load	215 tickets per eval

The whole demo run took about 17 minutes of GPU time, ≈ $1.60 at $5.49/h. The spike passed on MI300X (192 GB); on 13 Oct RunPod had no MI300X left, so the demo ran on an MI350X (gfx950, 288 GB) after re-running the training and serving checks there. No QLoRA, no bitsandbytes.

MARKET

Who pays first

Online shops

Ticket triage, order-intent detection, review moderation in their own languages. Pay per run: GPU time plus a margin. A run like the demo uses about $1.60 of GPU.

Regulated sectors

Banks, health and public bodies that cannot send texts to an external API. Self-hosted licence: the factory and the adapters run inside their network.

Agencies

Integrators who build support tooling for many small merchants and need a repeatable way to produce and prove a small model per client.

This slide is a plan, not traction: there are no customers yet.

ARCHITECTURE

Control plane decides, GPU worker executes

Control plane · Laravel 12

Livewire 3 UI and Tailwind v4, MariaDB, Horizon queues. Run state machine, stop rules, Claude failure analyst, public read-only reports that keep working when the GPU is off.

GPU worker · FastAPI

Async jobs: /generate, /train, /evaluate, /serve, /predict. Bound to localhost; reached only through an SSH tunnel with a bearer token. Artifacts under /data/runs/<run>/.

Open source under MIT. The worker API is a written contract, so the GPU side can move to any ROCm host. Tests run without a GPU: pytest with a fake vLLM, PHPUnit with a fake worker.

NEXT

Next steps

- **Harder evals:** a human-labelled slice next to the synthetic eval, so a 0.99 on clear cases is checked against messy real tickets.
- **Extraction tasks:** order numbers, products, amounts, not only classification.
- **Multi-tenant runs** on a shared GPU with a queue, and adapters per merchant.
- **Preference tuning (DPO)** once a classification adapter exists.
- **A marketplace of adapters** for common commerce tasks, each with its frozen eval and report.